

From Water Jars to Code: An Autobiography of Resilience and Technology

Table of Contents

Chapter 1: The Soil of Laikipia – Water Jars and Elephants.....	2
Chapter 2: The Bridge of Chereta – Awakening	5
Chapter 3: The Portal of Code – Initiation.....	8
Chapter 4: The Circumcision of Tech – Transformation	11
Chapter 5: The Craftsman’s Workshop – Application.....	14
Chapter 6: The Code of My Fathers – Synthesis	17
Chapter 7: The API to the Future – Vision.....	20

Chapter 1: The Soil of Laikipia – Water Jars and Elephants

Before I knew what a server was, I knew the weight of a full water jar. Before I debugged code, I navigated paths where elephants walked. *My journey into technology did not begin in a computer lab; it began here, on this earth.*

That path began before sunrise, when Muhotetu was still half-asleep and the world felt unfinished. The sky at 5 AM was neither dark nor light, just a stretched gray-blue canvas, as if God was still deciding what kind of day it would be. The cold bit softly at my fingers, but the red soil was already awake, breathing dust with every step.

Muhotetu, Laikipia, was not quiet. It only sounded quiet if you listened poorly. In the early mornings, the wind moved through dry shrubs like whispered conversations. Birds rehearsed their songs, testing notes. Far away, cows shifted, bells clinking like loose coins. The air smelled of dust, dung, and wood smoke from last night's fire, still clinging to the earth. When the sun finally rose, it did not arrive gently. It came with authority, warming the soil, sharpening shadows, reminding us that the day had work for us.

Our home was simple, but it was anchored. Mud walls, iron sheets, a small compound shaped by footsteps rather than fences. Life followed patterns that were never written down, but everyone knew them. Wake early. Fetch water. Tend animals. Prepare food. Repeat. It was a system that worked because everyone understood their role, even if no one ever named it that way.

My mother, Rahab Njoki, was the center of that system. Not loudly, not dramatically, but decisively. She carried the household the way some people carry faith quietly and without complaint. I remember one season when the rains failed completely. Crops dried in the ground before they could even disappoint us. People spoke in low voices then. Worry hung in the air heavier than smoke. One evening, after we realized the maize harvest would not come, my mother sat outside, stared at the land for a long time, then stood up and said, "We adjust." No tears. No panic. The next day, she began trading labor for food, reorganizing meals, stretching

what little we had. Watching her taught me something I wouldn't name until much later: *systems must adapt, or they collapse. Resilience is the first requirement of any functional code.*

My sister, Perpetual, was my constant companion. We shared chores, secrets, and a kind of silent understanding that didn't require many words. When we walked together to fetch water, we talked about small things who had the faster goat, whose sandals were wearing out but beneath that was an unspoken rule: we move together, we look out for each other. That may have been our family's motto, though no one ever said it aloud. Endure. Share. Move forward.

The water journey was the most defining ritual of my childhood. We carried yellow jerrycans *kibuyu* sometimes five liters, sometimes more, depending on how brave or foolish we felt that morning. The path stretched across open land, land that did not belong to us in the way a title deed defines ownership, but in the way familiarity does. We knew which stones shifted, which bushes hid thorns, which bends demanded caution.

Balancing the *kibuyu* was an art. Too far left, and your shoulder burned. Too far right, and your neck protested. I learned to walk with a certain rhythm, adjusting posture subconsciously, distributing weight efficiently. Years later, when I would optimize code or refactor logic to reduce load, I would feel a strange familiarity. *The body remembers lessons the mind forgets.*

Sometimes, the journey offered surprises. Elephants. Massive, silent, unbothered by our schedules. When we spotted them, everything stopped. No debate. No heroics. You waited. You hid. You rerouted. Fear sharpened awareness. You read signs quickly the direction of movement, the mood, the distance. An elephant on your path was not evil; it was simply there. You adjusted or you suffered consequences. Much later, bugs would feel the same way: unexpected, dangerous if ignored, neutral if understood.

On those walks, conversations drifted. Older women discussed prices, sickness, rumors. Children argued, laughed, complained about sore feet. But there were long stretches of silence too, filled only by footsteps and breath. In that silence, my mind worked. I counted steps. I calculated how long before the sun climbed higher. I wondered how to make the journey faster, easier. These were my first algorithms, though I didn't know the word. *If we leave earlier, we*

avoid heat. If we fill containers in a certain order, we wait less. If we fix a shoe with wire instead of waiting for money, we walk tomorrow. Problem. Constraint. Solution.

At home, scarcity sharpened creativity. One sack of maize had to last. Shoes were repaired, not replaced. Broken things were puzzles, not trash. I watched adults reason through problems aloud, testing ideas, discarding bad ones, committing to what worked. There was no romance in it, but there was rigor. Efficiency was survival.

Every so often, a flicker from another world appeared. A motorbike passing through, its engine sound cutting the air like a question. A visitor with a phone, light glowing from their palm like captured magic. I remember staring at a screen once, watching images move, feeling a quiet pull. Not envy exactly curiosity. Where did that world begin? How did it work? What rules governed it? I didn't yet want to escape Muhotetu, but I wanted to understand what lay beyond it. School gave names to some of my instincts, but the foundation had already been laid elsewhere. Discipline was waking before dawn. Debugging was rerouting around elephants. Systems thinking was understanding that if water did not come, nothing else worked. Community was sharing a path, a source, a burden.

I did not know it then, but I was being trained. The discipline to wake before dawn would become the discipline to debug code into the early hours. The patience to carry a heavy load for miles would become the patience to build a complex feature, line by line. The community that shared one water source would become the open-source community that shares code. The foundation was not soft; it was hard, baked by the Laikipia sun, shaped by necessity, and strong enough to carry everything that would come after.

Chapter 2: The Bridge of Chereta – Awakening

Chereta Secondary School did not have elephants. It had something else: silence. Not the silence of land stretching endlessly under the sky, but the silence of minds bent toward the same task. In Muhotetu, silence meant caution – you listened for danger, for movement, for change. In Chereta, silence meant focus. It meant order. It meant that something invisible but important was happening.

The journey to Chereta felt longer than the distance it covered. Leaving home that first time, I carried more than a box and a few clothes. I carried a whole way of thinking shaped by red soil and early mornings. When the school compound came into view, everything looked deliberate. Buildings aligned. Paths clear. Bells mounted on walls, waiting to dictate time itself. There was grass that was not trampled randomly but cut evenly, almost respectfully. This place was *designed*, not improvised.

What struck me most was the feeling of containment. At home, the horizon was always visible. In Chereta, the world felt enclosed by rules, by schedules, by expectations. It was what people called a “conducive environment,” though I did not yet have the language for why it felt that way. The absence of chaos was unsettling at first. There was no need to scan for elephants. No need to calculate water weight. The problems here were abstract, printed neatly on paper, timed by bells.

The first system I had to learn was time. Bells ruled everything. Wake-up bell. Prep bell. Class bell. Lunch bell. Evening prep. Lights out. In Muhotetu, time was guided by light, hunger, and necessity. Here, time was sliced into units and enforced. At first, I resisted it instinctively. But slowly, I recognized its power. Structure reduced uncertainty. It freed mental space. Once you trusted the system, you could focus entirely on the work inside it.

The timetable became my new map. Mathematics at this hour. English next. Sciences layered carefully through the week. Exams punctuated the term like checkpoints. I learned to plan backwards from exams, to allocate effort where it mattered most. This was another kind of algorithm: *given limited energy, how do you maximize outcome?* The logic felt familiar, even if the context was new.

Not everything came easily. Some subjects demanded a kind of abstract patience I had never practiced. Physics problems that existed only on paper. Chemistry equations that required faith in invisible reactions. But resilience was not new to me. When my mind grew tired, I remembered physical exhaustion the burn of shoulders under a water jar, the heat of midday walks. Studying tired was different, but not harder. Pain had trained me for persistence.

There were teachers who noticed this before I did. One in particular—quiet, observant—never praised loudly, but corrected carefully. When I performed well, he pushed further. When I struggled, he didn't lower the bar; he explained the system again, more clearly. He taught me that intelligence was not speed, but consistency. That discipline was not punishment, but freedom. For the first time, I saw how my background could translate into academic strength, not something to be hidden or overcome.

Friendships formed slowly. Many students came from places very different from Muhotetu. Their references, their assumptions, their confidence were shaped elsewhere. At first, I listened more than I spoke. I learned how others studied, how they revised, how they talked about “careers” as if those paths were already mapped. I didn't feel inferior, but I felt *behind*—like someone who had learned to run on rough terrain suddenly placed on a track with rules I hadn't memorized yet.

When motivation dipped, my mind traveled back home. I saw my mother's hands measuring maize carefully. I saw my sister lifting water containers without complaint. Those images carried weight. I was not studying only for myself. I was carrying expectation—not spoken, not demanded, but deeply felt. Failure here would not just be a bad grade. It would be a broken bridge.

KCSE loomed over everything. It was not just an exam; it was a gate. Teachers spoke of it with gravity. Students measured their worth against it constantly. In my mind, KCSE was a

port one ship, one departure. If you missed it, there was no easy reroute. I studied with urgency, but not panic. I had learned long ago that panic wastes energy.

The exam period itself passed in a blur of papers, invigilators, and silence thick enough to feel. Afterward came waiting. That was the hardest part. In Muhotetu, effort produced immediate outcomes you fetched water, you had water. Here, effort disappeared into envelopes and systems beyond your control. Waiting taught me humility.

When results came, they did not arrive quietly. They arrived with noise, celebration, comparison. I had passed. Not barely. Well enough to choose. That realization felt unreal. I had crossed the bridge successfully. But standing on the other side, I saw not clarity, but distance.

University appeared on the horizon, vast and undefined. Information Technology was the course I selected not because I fully understood it, but because it promised relevance, modernity, movement. I knew it was important. I did not yet know why. The logic of servers and systems was still hidden from me, like elephants beyond a bend in the path.

I had earned my passage. The bridge behind me was solid. But ahead was open terrain again different soil, different rules. The silence of Chereta had awakened something in me, but it had also revealed how much I still did not know. And so I stepped forward, carrying both confidence and uncertainty, ready to learn a new system once more.

Chapter 3: The Portal of Code – Initiation

My cousin Eva Wamuyu did not ask if I wanted to study Information Technology. She told me I would. It happened over a phone call that felt almost too casual for what it carried. No long introduction. No dramatic buildup. Just her voice, confident, already several steps ahead of me. “You should do IT,” she said. “That’s what you’ll study.”

There was a pause on my end. Not because I was considering it, but because my mind was scrambling to catch up. I remember holding the phone slightly away from my ear, as if distance might help me process the statement. In that moment, my future shifted quietly, without ceremony. It didn’t feel like excitement. It felt like being handed a responsibility I hadn’t asked for, a map written in a language I didn’t yet speak.

Eva lived in a different world. Not geographically distant, but mentally so far ahead that it felt like another era. Her world was made of laptops, fast conversations, ideas that moved quicker than I was used to. She spoke about technology the way people from home spoke about rain—with certainty, inevitability. To her, IT wasn’t just a course. It was a direction. A lever that could move lives.

I didn’t doubt her intelligence or her intent. What I doubted was myself. After the call ended, the silence in the room pressed in. My thoughts became loud. I replayed her words over and over, hoping they might rearrange themselves into something less intimidating. IT. Computers. Code. I had never touched a computer beyond the basics. I had no background. No exposure. No foundation. My mind immediately began negotiating.

“I’ll change the course once I get there,” I told myself. Just a placeholder. A temporary decision. A way to cross the bridge without committing to the destination. That thought became my comfort. It allowed me to move forward without fully facing the fear. I convinced myself that choosing IT was not acceptance it was delay. I would arrive, assess, then pivot. That way, if I failed, it wouldn’t be because I was incapable. It would simply be because the course was wrong.

When I first walked into Dedan Kimathi University of Technology, the feeling was different from Chereta. Chereta had been quiet, structured, controlled. DeKUT was alive. Conversations spilled everywhere. People spoke quickly, confidently, using words that seemed to stack on top of each other without pause: “Programming.”, “Python.”, “Backend.”, “Frontend.”, “Frameworks.” They sounded like spells from a book I had never read.

I moved through campus with the same instinct I had learned in new environments before observe first, speak later. But this time, observation didn’t calm me. It amplified the gap. At Chereta, I had felt behind but capable. Here, I felt invisible. Like everyone else had received a memo I had missed. The first lectures confirmed my fear. Slides filled with terms that assumed familiarity. Lecturers moved quickly, not out of cruelty, but expectation. This was a place where prior exposure was currency. I sat there copying notes diligently, even when I didn’t understand them, trusting that understanding might come later if I just captured the words. It didn’t come easily.

In labs, I watched screens fill with lines of text that meant nothing to me. Others typed confidently, occasionally frustrated, but always certain they were operating within something they understood. I felt like an imposter who had wandered into the wrong room and was hoping no one would ask questions. And yet this was not my first time being out of place.

The difference was that in Muhotetu and Chereta, effort always translated into progress. Here, effort sometimes felt like shouting into a void. I studied harder, but clarity came slowly. I began to dread moments when lecturers asked questions. Not because I didn’t care, but because I cared too much.

Then, quietly, without announcement, a crack appeared.

It was a simple thing. A basic program. A few lines of code. I followed instructions mechanically, not fully understanding why each step mattered. I typed exactly what was written. I pressed run. Something happened. The screen responded. Not with an error, but with output. Text appeared where nothing had been before. It wasn’t impressive. It wasn’t complex. But it worked.

That moment didn’t bring joy. It brought relief. A subtle but important shift. The fog didn’t lift completely, but it thinned. For the first time, I saw that this world was not magic. It was

logic layered, precise, unforgiving, but learnable. Still, the doubt didn't disappear. "I will change courses," the voice returned whenever things felt heavy. But each time I considered it seriously, something held me back. Part of it was stubbornness. I had crossed too many bridges to turn back at the first sign of discomfort. Part of it was fear not of IT, but of retreat. Going back felt heavier than pushing forward. And part of it, though I didn't admit it then, was trust. Eva had seen something I couldn't yet see. And she had never spoken carelessly. So I stayed.

I told myself I would give it time. One semester. One year. Long enough to be sure. I stopped framing confusion as failure and began treating it as a phase. I learned to sit with not knowing, the same way I had learned to sit with hunger or fatigue acknowledging it without letting it define the outcome.

The internal attachment period came quietly. It wasn't dramatic. It didn't immediately validate my choice or erase my fears. But it introduced something new: context. Theories began to connect to practice. Concepts that once felt abstract gained edges. I was still slow, still unsure, but no longer completely lost. More importantly, I noticed something unexpected. I was curious. Not confident. Not skilled. But curious. I wanted to know why things worked. I wanted to fix what broke. I wanted to understand systems, not just survive them. That curiosity was faint, easily drowned by doubt, but it was there and it was persistent.

As the Teach2Give attachment approached, nervousness returned with force. This was no longer internal. This was the real world. Real expectations. Real scrutiny. I felt like I was standing at the edge of something irreversible. I thought back to the phone call. To the prophecy delivered without ceremony. To the map I couldn't read. I realized that I hadn't been dragged here. I had walked. Hesitantly, reluctantly, but deliberately. I was about to step through a portal. And for the first time, alongside fear, there was something else. A readiness I didn't yet trust but could no longer deny.

Chapter 4: The Circumcision of Tech – Transformation

Teach2Give was not a training program. It was a ritual. A deliberate, intense, almost confrontational stripping away of who I thought I was, to make room for who I needed to become. University had introduced me to ideas, to theories, to the vocabulary of technology. Teach2Give introduced me to myself under pressure, under expectation, under fire. The confusion I had known at DeKUT was abstract and forgiving. This was physical. This was time-bound. This was real.

In Kikuyu tradition, *Irua* is not explained away. You are not comforted through it. You are expected to endure, to remain still, to accept pain as the price of transformation. Teach2Give felt exactly like that. I arrived on the first day carrying quiet confidence and loud fear. The space was different from a lecture hall. No rows of passive listeners. No hiding at the back. The air itself felt charged, as if everyone in the room knew they were about to be tested. The instructors didn't introduce themselves with long biographies. They spoke with precision, economy. Their presence communicated something unmistakable: *standards lived here*.

I looked around at the other trainees. Some seemed relaxed, already fluent in the language of code. Others, like me, wore the tension of anticipation. I asked myself if I was prepared. The honest answer was no but I had learned by then that preparation often comes after commitment, not before. The days became long almost immediately. Mornings blurred into evenings. Code replaced small talk. Problems stacked on top of each other without pause. There was no illusion of gradual ease. You either kept up, or you confronted yourself. This was where the pain lived.

Not dramatic pain, but relentless friction. Bugs that refused to move. Errors that mocked me with red lines and cryptic messages. Logic that collapsed one step before success. I remember staring at my screen late into the evening, convinced I had broken something fundamental not just the code, but myself. Every instinct urged me to step away, to defer, to say *later*. But Teach2Give had no later. There was only now, and now demanded everything. This was the cut. Each unresolved bug felt personal. Each failed build felt like exposure. There were moments when frustration tightened my chest, when the simplicity of the water paths of

Muhotetu seemed infinitely kinder than this invisible resistance. At least there, effort translated directly into movement. Here, effort sometimes produced nothing at all. But just as in *Irua*, the pain was not arbitrary. It was surgical. Slowly, patterns emerged. I learned that precision mattered more than speed. That assumptions were dangerous. That computers did not forgive carelessness, but they rewarded clarity. I learned to sit with problems without panicking. To break them down. To trace them back to their origin. This was not just coding. It was discipline.

The mentors our elders in this rite played their role with quiet authority. They did not rescue us from difficulty. They guided us through it. A question here. A correction there. Sometimes just silence, allowing us to struggle long enough to grow. They taught me something more valuable than syntax: responsibility. Your code was your word. If it failed, you owned it. If it succeeded, you understood why.

One instructor in particular had a way of looking at broken code that felt almost ceremonial. He wouldn't touch the keyboard immediately. He would ask, "What did you intend this to do?" That question forced me to confront the gap between thought and execution. Over time, that gap narrowed. Then came the first real build. Not an exercise. Not a demo. A functional application. Something meant to be used. Something that solved an actual problem. I remember the moment the task was introduced. The weight of it settled heavily. This was no longer about learning concepts. This was about delivery.

The process was brutal and beautiful. Planning. Designing. Writing backend logic. Connecting routes. Handling errors. Watching things fail in ways I hadn't imagined. Fixing one issue only to reveal three more. There were nights when I questioned my capacity entirely. But something had shifted by then. I no longer interpreted difficulty as a signal to stop. It had become a signal to focus.

The breakthrough didn't arrive dramatically. It came quietly, almost suspiciously. The application loaded. The requests worked. Data moved as it should. The screen responded correctly. I remember sitting back, not celebrating immediately, as if afraid to disturb the moment. I refreshed the page again. It still worked. That was the moment I *stood still*. Not physically, but internally. Something settled. For the first time, I wasn't borrowing confidence

from my cousin's belief or my family's sacrifices. It was mine. Earned. Tested. I didn't announce it to anyone, but I knew: *I could build.*

After that project, I looked at a computer screen differently. It was no longer an object of intimidation. It was a medium. Clay. Wood. Something shaped by intention and skill. I introduced myself differently too not out loud, but within. I stopped saying "I am learning tech" and started thinking "I am a developer in formation."

The foundation Teach2Give gave me went far beyond frameworks like Express.js or languages like TypeScript. It gave me a mindset anchored in accountability. The confidence that learning was not about prior exposure, but sustained effort. The understanding that confusion was not failure it was the doorway. This was *Irua*.

Just as a boy endures pain to gain the rights and responsibilities of manhood, I endured the intensity of Teach2Give to gain entry into the world of software engineering. The curiosity from university hardened into craft. The fear softened into respect. I had been cut away from the version of myself that waited to be told what to do, and welcomed into one that could build.

When the attachment ended, I walked out different. Not loud. Not arrogant. But grounded. I carried a new name, even if only in my own mind: backend-focused, full-stack software engineer. The ritual was complete. And like all true rites of passage, there was no turning back.

Chapter 5: The Craftsman's Workshop – Application

The DeKUT Startup & Incubation Center did not feel like a classroom, and it did not feel like a ritual ground either. It felt like a workshop. A place where ideas were expected to leave fingerprints. Where theories were not debated endlessly but tested, bent, sometimes broken, and rebuilt. The air carried a different kind of pressure not the sharp intensity of Teach2Give, but the steady weight of expectation. Here, you were not proving that you could learn. You were proving that you could build.

Walking into the Startup Center for the first time, I sensed immediately that this was a transition. Teach2Give had forged me internally. This place would test me externally. Laptops were open everywhere, conversations jumped between business models and database schemas, whiteboards were layered with half-erased ideas. People spoke in the language of possibility, but with the urgency of deadlines. Time mattered here because *users* mattered. This was where my identity as a developer would either solidify or fracture. The difference from Teach2Give was subtle but profound. There, the struggle was personal. Each bug was a private trial. Here, the struggle was shared. Your mistake could slow a team. Your breakthrough could unblock everyone. Collaboration was no longer a soft skill; it was structural. You were a component in a larger system. It was in this environment that **Skill Match AI** was born. The idea did not arrive fully formed. It emerged gradually, shaped by conversations, frustrations, and lived experience. We spoke often about opportunity who gets access to it, and who doesn't. About talented people who never find the right door, not because they lack ability, but because the system fails to recognize them. As those discussions unfolded, I realized something uncomfortable and clarifying: I was not theorizing about this problem. I had lived it. Skill Match AI became our commission. At its core, the project was about connection. Bridging the gap between skill and opportunity, between potential and visibility. In many ways, it was a response to the confusion I had felt standing at the edge of university, holding grades but lacking direction. We wanted to build a system that could reduce that gap not through guesswork, but through logic. This time, I was not just writing code. I was designing an engine. I took responsibility for the backend architecture, and I approached it the way a craftsman approaches a blueprint. The first question was not “what tools do I know?” but “what does this system need to do, reliably, at scale?” I chose Node.js and TypeScript as the

foundation not out of habit, but intention. I needed predictability, structure, and safety. TypeScript gave me the discipline I had learned to value: clear contracts, fewer assumptions, more trust in the system.

The backend became the engine room. Quiet. Invisible to the user. Essential. I designed RESTful APIs as deliberate protocols of conversation. Each endpoint was a sentence with a purpose: *submit skills*, *fetch matches*, *authenticate users*, *manage roles*. The frontend would speak; the backend would listen, reason, and respond. Cleanly. Predictably. This was not about cleverness. It was about clarity.

PostgreSQL became the memory of the system the organized archive where relationships mattered. Skills related to users. Users related to roles. Opportunities related to requirements. I used TypeORM not as a convenience, but as a translator between logic and persistence. It allowed me to think in objects while respecting the rigor of relational data. Tables were not just storage; they were agreements.

One of the most defining architectural decisions was implementing role-based dashboards. Job seekers and employers were interacting with the same system, but they needed different truths surfaced. This forced me to think deeply about authorization, data access, and perspective. The same data, filtered through different lenses. That idea stayed with me it mirrored life itself more than I expected.

Building Skill Match AI was not a solo endeavor. It was ***Gĩĩo***. In Kikuyu culture, *Gĩĩo* is communal labor. People come together to build a house, harvest a field, or solve a problem that is too large for one person. The Startup Center was exactly that, translated into modern tools. Git and GitHub were our meeting ground. Pull requests replaced handshakes. Code reviews replaced spoken corrections. But the spirit was the same.

I remember a particularly stubborn bug that threatened to derail a core feature. Matching logic that worked in isolation collapsed under real data. For hours, we circled the problem, each of us bringing a different perspective. One teammate questioned assumptions. Another traced queries. I re-examined the algorithm itself. No single hero moment. Just collective persistence.

When the fix finally emerged, it felt communal. Not triumphant, but grounding. We had earned that solution together. That moment taught me something critical: mastery is not isolation. It

is contribution. The most significant technical challenge I faced came from performance and correctness colliding. Early on, our matching queries were accurate but slow. The system *worked* until it didn't. Under realistic data volumes, response times degraded. This forced me into deeper waters. Query optimization. Indexing strategies. Rethinking logic placement between database and application layer.

Debugging here was different from Teach2Give. The question was no longer “why doesn't this work?” but “why doesn't this *scale*?” That distinction marked a new level of thinking. I learned to profile, to measure, to respect the cost of decisions. I learned that elegant code that ignores reality is still broken.

When the platform finally reached a stable, working state, the feeling was not explosive. It was steady. Quiet pride. I watched users interact with a system I had helped architect from nothing. Requests flowed. Data moved. Matches appeared. The logic held.

In that moment, I understood something deeply personal: *I was building the tool I once needed.*

Skill Match AI was not perfect, but it was real. And real systems carry responsibility. Seeing it function in a real-world context anchored my identity in a way no certificate could. I was no longer rehearsing the role of a software engineer. I was practicing it.

As the project wrapped up, I looked at my tools Node.js, TypeScript, PostgreSQL, APIs not as abstract technologies, but as extensions of my thinking. They were no longer foreign. They were familiar, dependable. I had crossed another threshold.

From initiate to craftsman. From learning to application. From potential to proof.

And standing in that workshop, surrounded by unfinished ideas and future builds, I knew I was ready for what came next.

Chapter 6: The Code of My Fathers – Synthesis

For a long time, I believed my culture lived behind me. In stories told at dusk, in ceremonies I watched as a child, in customs that seemed fixed in another era. Technology, by contrast, felt forward-facing clean, rational, unburdened by ritual. Code compiled or it didn't. Systems scaled or they failed. There was no room, I thought, for tradition in that world.

I was wrong. Only after building systems of my own did I begin to recognize the deeper architecture shaping how I think. The habits that carried me through confusion. The patience with structured difficulty. The respect for process, community, and commitment. When I looked closely, I realized these were not values I learned from documentation or lectures. They were inherited. Cultural. Quietly embedded long before I ever touched a keyboard.

I returned to culture not out of nostalgia, but out of necessity. I wanted to understand *why* I work the way I do, not just *how*. And when I examined the Kikuyu frameworks I had named earlier *Irua* and *Ruracio* they revealed themselves not as relics, but as design patterns. Tested systems for managing transition, responsibility, and trust.

***Irua*, when stripped of symbolism and fear, is a study in purposeful transformation.** It is not pain for its own sake. It is pain with direction. The boy who enters the ritual is not meant to remain who he was. The discomfort is deliberate, bounded, and socially meaningful. It marks the end of one identity and the emergence of another.

What struck me, looking back, was how closely this mirrored my experience learning to code seriously especially at Teach2Give. The struggle was not random. The long nights, the walls I hit, the frustration of bugs that refused to yield were not signs of failure. They were the process. Necessary resistance. Without them, the identity of “software engineer” would have been hollow.

Irua also insists on witnesses. Transformation does not happen in isolation. The community watches. It acknowledges the change. That public recognition matters. In tech, I saw this in presentations, code reviews, demos. When Skill Match AI was deployed and used, when peers and mentors interacted with something I had built, the identity shift was affirmed. Not internally assumed, but externally validated.

And with that recognition came responsibility. In Kikuyu society, a circumcised man is accountable to family, to community, to conduct. In engineering, deploying a system carries the same weight. Once code is live, it affects others. Errors matter. Reliability matters. You are no longer experimenting in private. You are responsible for outcomes.

Seen through this lens, Teach2Give was not just training. It was *Irua*. A structured ordeal designed to produce capability, not comfort.

If Irua explains how one becomes, Ruracio explains how one commits.

Dowry, often misunderstood as a transaction, is in reality a complex protocol. It is not a sale of a woman. It is a formal acknowledgment that relationships carry obligations. That commitment must be demonstrated, not declared casually. That families—systems larger than individuals are entering a covenant.

What fascinates me now is the structure. *Ruracio* follows rules. There are stages, negotiations, shared understanding. Nothing is rushed. Nothing is assumed. Each side clarifies expectations. Trust is built through process, not sentiment.

In software, I see this everywhere. An API contract is not just technical documentation. It is a promise. If I expose this endpoint, I guarantee certain behavior. If you depend on it, you trust that guarantee. Breaking it has consequences. Designing systems without this respect for structured agreement leads to fragile products and broken relationships.

During the Skill Match AI project, we negotiated features, timelines, and responsibilities constantly. Nothing meaningful was built unilaterally. Every decision required alignment. In that sense, the project was a form of modern *Ruracio*—a careful, respectful negotiation to create something durable.

Ruracio also builds bridges. It binds families into long-term relationships. Similarly, good software does not just solve a problem once; it creates ongoing trust between users and systems. People return because they believe the platform will behave responsibly.

These realizations forced me to confront another essential layer: gender **Traditional Kikuyu practices, like many cultural systems, were shaped in contexts that privileged male**

authority. It would be dishonest—and intellectually lazy—to ignore that. Women’s roles were often constrained, their agency mediated through family structures dominated by men.

And yet, my own life complicates any simplistic reading. The strongest system architect I have ever known is my mother, Rahab. She did not write code, but she managed scarcity, coordinated resources, enforced discipline, and sustained hope under conditions that would break many engineers. If my family was a system, she was its core logic.

And the person who redirected my future with a single phone call Eva was a woman. She saw potential where I saw confusion. She acted with authority and clarity. Not as a suggestion, but as a directive grounded in belief.

These women were not peripheral to my story. They were central. This forces a necessary conclusion: **culture evolves, or it collapses.** Just as technology must adapt to new realities, cultural frameworks must be reinterpreted to remain ethical and relevant. The collaborative spirit of *Gĩtĩo* when properly understood is not exclusive. It assumes contribution based on ability, not gender. It values participation, not hierarchy.

Modern engineering, at its best, reflects this evolution. Inclusive teams build better systems. Diverse perspectives expose blind spots. Exclusion is not tradition preserved; it is capacity wasted. **When I synthesize all this, I see how deeply culture informs my engineering practice.** I approach difficult problems with patience because I understand transformative pain. I respect specifications and contracts because I value structured commitment. I treat collaboration as communal labor, not competition. I see software not as cold logic, but as a social covenant—an agreement between people mediated by code.

Debugging is no longer just an annoyance. It is the cut that reveals weakness so strength can form. Teamwork is not a corporate buzzword. It is *Gĩtĩo* with modern tools. Deployment is not the end of work. It is the beginning of responsibility.

I am not torn between tradition and technology. I am shaped by both. My ancestors engineered social systems resilient enough to endure generations. I engineer digital systems meant to serve communities today. The materials have changed. The principles have not. And in that continuity, I have found coherence not conflict.

Chapter 7: The API to the Future – Vision

I have come to understand that technology is never neutral. Code carries the fingerprints of the life that writes it. I do not write in a vacuum; I write with memory. With weight. With a quiet obligation shaped long before I knew what an API was. Every line of logic I construct stands on paths once walked with a water jar balanced on my head, on evenings when scarcity forced invention, on voices that believed in me before I could explain what I was becoming.

That is the debt I carry not financial, but moral. Meaningful. I was given access to systems that many in my community never see. Education, connectivity, tools of abstraction. That access obligates me. Not to romanticize poverty or “give back” in a shallow sense, but to **build forward** to use what I have learned to design systems that widen paths rather than narrow them.

My responsibility, as I see it, is not simply to be a competent engineer. It is to be a **translator**. To stand between worlds and make them interoperable. The challenges that shaped me are not theoretical. They are persistent. The digital divide is not just about internet access; it is about relevance, usability, and trust. Many platforms assume literacy, stable connectivity, and familiarity with abstract interfaces. Rural communities often have none of these, yet they are rich in knowledge, labor, and potential.

One vision I carry is an **offline-first agricultural advisory system** built for farmers in arid and semi-arid regions like Laikipia. A platform that works on basic smartphones, syncing when connectivity appears, offering localized, seasonal advice on crops, livestock, and weather patterns. Not text-heavy dashboards, but voice-based prompts in local languages. A system that respects how people actually live, not how designers imagine they should.

Another is an **evolution of Skill Match AI** not just a web platform for urban job seekers, but a simplified, voice-assisted matching service for informal workers. Artisans, casual laborers, young people with skills but no formal certificates. A system where a person can describe what they can do, verbally, and be matched with opportunity without needing to navigate complex forms. Matching human capability to demand should not require fluency in English or comfort with keyboards.

These are not ideas born from trend analysis. **They come from lived gaps.** But vision without method is just ambition. What differentiates my approach is not only *what* I want to build, but *how*. I intend to practice **culturally informed engineering**. That means applying the patterns I now recognize. *Gĩĩo* will not just be a metaphor. It will be a process. Community involvement from the start co-design, testing, feedback loops that treat users as contributors, not endpoints. Just as communal labor requires coordination and mutual accountability, so does sustainable software development.

Irua reminds me that adoption has a cost. New technology introduces discomfort. Resistance. Confusion. That pain should not be dismissed. It should be anticipated and guided. Training, gradual rollout, patient iteration these are not inefficiencies; they are rites of passage. Respecting that makes systems endure.

Ruracio teaches commitment. I do not want to build disposable products. I want to build platforms with clear contracts ethical, technical, and social. Systems that earn trust over time because they behave predictably and responsibly. Where users understand what is expected, and developers honor those expectations.

Gender is not an afterthought in this vision. It is foundational. Too many systems fail because they treat women as a subset rather than central users. In many Kenyan communities, women are the primary managers of resources water, food, household economies. Designing systems without them in mind is not neutral; it is negligent.

I commit to designing with women from the beginning. To testing assumptions. To ensuring accessibility, safety, and agency. And beyond design, I see mentorship as obligation. I did not choose my cousin's role in my life, but I can choose to play that role for others. To point, firmly, toward horizons unseen. Especially for young women who have ability but lack exposure or confidence.

Legacy, for me, is not a résumé. It is resonance. I want my career to mean that somewhere, a young person from a place like Muhotetu finds a path clearer because a system I helped build existed. That a farmer makes a better decision. That a skilled worker finds dignity in opportunity. That technology feels less foreign, less hostile, more like a tool shaped for their hands.

If I am remembered, I want it to be as someone who built bridges—between rural and digital, tradition and innovation, logic and humanity. And so the story comes full circle. The boy who once carried water for kilometers now carries a laptop. The weight is different, but the purpose is the same. To sustain. To enable. To make life a little more navigable for those who come after.

I am the son of Rahab, shaped by resilience. A product of *Irua* and *Ruracio*, trained in transition and commitment. A backend-focused full-stack software engineer, fluent in systems and stories. A community builder who understands that code, at its best, is an act of service